

Relatório de Aprendizagem

Sistema RAG Distribuído via Tailscale — Tema 5

Aluno: Davi Maciel Cavalcante **Disciplina:** Engenharia de Contexto e Harness Engineering aplicada à Programação Distribuída e Paralela **Instituição:** CESUPA — 2º bimestre de 2026 **Período relatado:** 09/05/2026 a 25/05/2026

Sumário

1. Introdução
2. Contribuição técnica
3. Aprendizados técnicos 3.1 Bloco B1 — Fundação 3.2 Bloco B2 — Recuperação, rerank e cache 3.3 Bloco B3 — Resiliência, observabilidade e modo distribuído
4. Aprendizados de processo
5. Conclusão

1. Introdução

Este relatório documenta o que aprendi ao longo do projeto do Tema 5 da disciplina de Programação Distribuída e Paralela, em que desenvolvi, com dois colegas, um sistema de *Retrieval-Augmented Generation* (RAG) distribuído em três PCs físicos conectados via Tailscale, sem custos de nuvem. Procuro registrar tanto os ganhos técnicos — arquitetura assíncrona em Python, mensageria com RabbitMQ, engenharia de contexto para LLMs locais e observabilidade — quanto os de processo, em especial sobre como colaborar com uma IA assistente preservando a autoria do código. Os trechos a seguir são uma síntese reflexiva: privilegio o que ainda não estava estabilizado em mim quando comecei o projeto.

2. Contribuição técnica

Fui responsável pela fundação do projeto e por boa parte da evolução dela: o scaffold Python com uv e os gates de qualidade (ruff, mypy --strict); os módulos compartilhados (config, logging, messaging, ollama_client, cache, session); o gateway FastAPI com endpoints /ingest, /query e /health no padrão RPC sobre AMQP; o pipeline de ingestão (parser → chunker recursivo → embedding → upsert no Qdrant); o pipeline de query (embedding → retrieval → rerank → geração via Ollama com *function calling* para citações); a infraestrutura inicial em Docker Compose e a migração posterior para Terraform no modo distribuído; a observabilidade com Prometheus, Grafana e Loki; e os mecanismos de tolerância a falhas (DLX/DLQ com retry contado e fallback degraded mode). Contribuí também com a documentação técnica em docs/, incluindo a declaração pública de uso de IA exigida pelo enunciado.

3. Aprendizados técnicos

3.1 Bloco B1 — Fundação

- **Tooling Python moderno:** usar uv no lugar de pip/venv consolidou reprodutibilidade por padrão; ruff substituiu quatro ferramentas que eu mantinha separadas; e mypy --strict mostrou que type hints cobrem caminhos que testes de runtime nem exercitam.
- **Async-first em Python:** a tríade httpx.AsyncClient + tenacity (retry com backoff exponencial) + aio-pika (RabbitMQ) ensinou que await é operador semântico, não marcador de chamada importante. Coroutine sem await é “promessa não cumprida” — fonte de bugs silenciosos em que a mensagem nunca chegava ao broker, mas o endpoint respondia 200.
- **Estratégia de chunking no RAG:** Chunks de ~800 tokens com 120 de overlap maximizam a “afiação semântica” do embedding sem mutilar frases nas fronteiras.

- **Engenharia de contexto:** o `prompt_builder` consolidou a noção de que o `prompt` é o único canal entre o LLM e o conhecimento recuperado. Separar política (*system*) de payload (*user*) com etiqueta de fonte em cada bloco é o que viabiliza citação rastreável e responsabiliza o modelo por não alucinar.

3.2 Bloco B2 — Recuperação, rerank e cache

- **Pipeline parallelism:** partir a fila de ingestão em duas etapas (`documents` → `chunks`) traz uma maior eficiência na utilização de recursos computacionais. Parsing é leve e embedding é I/O-carro, então cada estágio escala independente conforme o gargalo real.
- **Cache distribuído em duas camadas:** L1 (embedding da query, TTL longo) e L2 (resposta completa, TTL curto, chaveada por query + IDs recuperados) curto-circuitam fases diferentes do pipeline e fazem sentido por razões diferentes — TTL longo onde o resultado é determinístico, curto onde depende do estado do corpus.
- **Cross-encoder no rerank:** descobri que rerank é o segundo estágio do retrieval por construção, não opcional. Top-20 vetorial barato seguido de top-5 por cross-encoder caro entrega a melhor relação custo-benefício do pipeline, no contexto do nosso trabalho.
- **Function calling para saída estruturada:** usar `tool_calls` em vez de regex em texto livre transformou a citação de “melhor esforço” em contrato — o modelo emite JSON estruturado que vira o campo `citations` da resposta.

3.3 Bloco B3 — Resiliência, observabilidade e modo distribuído

- **DLX/DLQ é contrato em três peças:** exchange *dead-letter*, filas DLQ correspondentes, e argumentos `x-dead-letter-*` nas filas principais. Faltar uma peça quebra o todo silenciosamente.
- **Degradar é melhor que quebrar:** o gateway preferir retornar chunks brutos com `[degraded mode]` em vez de 504 vazio quando o worker trava foi decisão consciente. Parti do entendimento que a aplicação não deveria quebrar se um worker apresentar mal funcionamento.
- **Observabilidade é design, não digitação:** `structlog` com `correlation_id` propagado via `contextvars` reconstitui o caminho de uma requisição entre gateway → fila → worker → Ollama. Sem isso, depurar o sistema seria quase impossível.
- **Modo distribuído real:** na migração do Docker Compose (Modo 1) para Terraform com três PCs via Tailscale (Modo 2), aprendi que o atributo `keep_locally = true` do provider Docker bloqueia rebuilds — imagens antigas continuam rodando até serem apagadas manualmente. Também aprendi que a paridade entre o `compose` e o `terraform` precisa ser explícita: omitir o bloco `gpu = "all"` no `terraform` fez o Ollama subir em CPU pura.

4. Aprendizados de processo

- **Spec antes de código:** ler o spec do bloco e o plano vigente antes de escrever uma linha evitou retrabalho sobre decisões já tomadas. A documentação densa precede o código por design neste projeto, e seguir essa ordem economiza horas.
- **TDD:** TDD para módulos isolados com lógica não trivial (*chunking*, *cache*, *prompt builder*); *smoke* ponta-a-ponta para integrações; *chaos test* para tolerância a falhas.
- **IA como parceiro, não gerador:** na *skill code-partner*, a IA entregava o contrato executável (testes, *scaffolding*, sinalização de API em comentário) e eu escrevia o miolo da lógica de produção. Quando aceitei mover essa fronteira de propósito — por exemplo, pedindo que ela escrevesse o boilerplate de um endpoint — declarei explicitamente.

5. Conclusão

Em pouco mais de duas semanas consegui entregar um sistema RAG distribuído, com observabilidade, tolerância a falhas e infraestrutura como código. Os erros documentados no caminho — `await`

esquecido, polaridade invertida em condicional, GPU não declarada no terraform — foram, em retrospecto, a parte mais educativa do processo: cada um consolidou um conceito que eu sabia achava que sabia.